INDEX

1. Introduction.....(1 – 4)

1.1 Introduction of the System
1.1.1 Project Title
1.1.2 Category
1.1.3 Overview
1.2 Objectives of the System
1.3 Scope of the System
1.4 Structure of the System
1.5 System Architecture
1.6 End Users
1.7 Software/Hardware Used for Development
1.8 Software/Hardware Required for Implementation

2. Software Requirements Specification (SRS)..... (5 – 8)

2.1 Introduction
2.2 Overall Description
2.2.1 Product Perspective
2.2.2 Product Functions
2.2.3 User Characteristics
2.2.4 General Constraints
2.3 Special Requirements
2.4 Functional Requirements
2.4.1 Module 1: Authentication
2.4.2 Module 2: Student Management
2.4.3 Module 3: Course Management
2.4.4 Module 4: Result Management
2.4.5 Module 5: Student View Result
2.4.6 Module 6: Report Generation
2.5 Design Constraints
2.6 System Attributes
2.7 Other Requirements

3. System Design.....(9 – 13)

3.1 Introduction
3.2 Assumptions and Constraints
3.3 Functional Decomposition
3.4 Description of Programs
3.4.1 Context Flow Diagram
3.4.2 Data Flow Diagrams (Level 0, 1, 2)
3.5 Description of Components

4. Database Design ..... (14 – 18)

4.1 Introduction
4.2 Purpose and Scope
4.3 Table Definition
4.4 ER Diagram

5. Detailed Design.....(19 – 23)

5.1 Introduction
5.2 Structure of the Software Package
5.3 Modular Decomposition
5.3.1 Module 1: Login Module
5.3.2 Module 2: Student Management Module
5.3.3 Module 3: Course Management Module
5.3.4 Module 4: Result Management Module
5.3.5 Module 5: Student Result View Module
5.3.6 Module 6: Report Generation Module

6. Program Code Listing ..... (24 – 30)

6.1 Database Connection
6.2 Authorization / Authentication
6.3 Home/Dashboard
6.4 Student Management
6.5 Course Management
6.6 Result Management
6.7 View Results
6.8 Exam Information
6.9 Error Handling
6.10 Application Entry Point

7. User Interface.....(31 – 38)

7.1 Login Page
7.2 Home/ Dashboard
7.3 Navigation Menu
7.4 Data Management
7.5 Validation
7.6 View
7.7 On-Screen Reports
7.8 Alerts
7.9 Error Messages

8. Testing.....(39 – 43)
8.1 Introduction
8.1.1 Unit Testing
8.1.2 Integration Testing
8.1.3 System Testing
8.2 Test Reports
9. Conclusion .....(44 – 45)
10. Limitations ..... (46 – 47)
11. Scope for Enhancement.....(48 – 49)
12. Abbreviations and Acronyms..... (50 – 51)
13. Bibliography / References.....(52 – 53)

Chapter -1
Introduction

1. Introduction

1.1 Introduction of the System

1.1.1 Project Title

CampusNest - SMART SCHOOL ERP SYSTEM

1.1.2 Category

Web Development

1.1.3 Overview

The system is divided into multiple modules such as login, admin dashboard, faculty management, student management, attendance tracking, marks entry, and material sharing. The admin can manage users, courses, and subjects. Faculty can mark attendance, enter marks, and schedule classes. Students can view their academic performance, attendance, and study materials through an individual portal. The application is built using the MERN stack (MongoDB, Express.js, React, Node.js), ensuring secure and efficient data handling. The system is designed with scalability in mind, allowing future enhancements

1.2 Objectives of the System

- To provide a user-friendly system for managing student and academic data.
- To automate attendance, marks and student records.
- To ensure secure role-based access for users.
- To help students easily view their performance online.
- To develop a scalable system for educational institutions.

1.3 Scope of the System

- Suitable for schools, colleges and other educational institutions.
- Web-based application accessible through desktop and networks.
- Supports student, course, and result management with role-based access.
- Can be extended with features like mobile apps, notifications and reports.

1.4 Structure of the System

1. **Login Module** – Secure login for admin and staff with role-based access.
2. **Student Management Module** – Add, update, and view student details.
3. **Course Management Module** – Add, update, and manage courses and subjects.
4. **Result Module** – Enter, calculate, and display student results.
5. **Report Viewer** – Generates and exports result summaries for printing or PDF.

1.5 System Architecture

The system architecture is designed to provide a secure and efficient web-based platform for managing school operations. It integrates role-based access, student and course management, attendance, fee handling and result processing with a centralized database. This ensures smooth interaction between users and reliable data management.

Main Layers:

1. Users

- Includes Admin, Faculty and Students.
- Access is provided after authentication.

2. Authentication

- Secure login using username and password.
- Once authenticated, they are directed to their respective interfaces based on their role.

3. Admin UI

- Admin can manage students, courses, and results.
- Admin can generate and view result reports.

4. Student UI

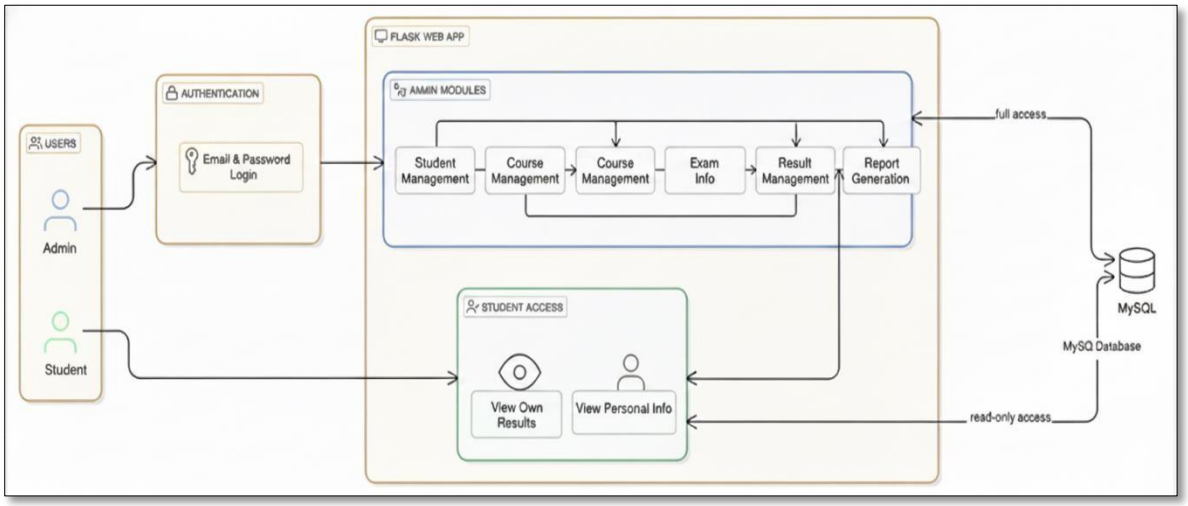
- Students can view their individual results and academic performance.
- Students have read-only access; they cannot modify any data.

5. Result Module

- Handles entry, calculation, and display of student results.
- Allows search using student details.

6. Database (MongoDB)

- Stores all student, course, and result information securely.
- Admin has full access, while students have restricted, view-only access.



1.6 End Users

- **Admin** – Responsible for managing students, courses, results, and reports.
- **Faculty** – Handles attendance, enters marks and updates student records.
- **Students** – Can view their individual academic results and performance.

1.7 Software/Hardware Used for Development

Software:

- Python 3.x with React framework.
- MongoDB database.
- HTML, CSS, Bootstrap for front-end.
- VS Code or any Python IDE.

Hardware:

- Desktop/Laptop (Windows or Linux OS).
- Minimum Intel Core i3 processor with 4 GB RAM.
- 500 GB or more storage.

1.8 Software/Hardware Required for Implementation

Software:

- Same stack as development (Node.js, MongoDB, React).
- Web browser for accessing the application .

Hardware:

- Any computer with internet access for web interface.
- Minimum Intel Core i3 processor with 4 GB RAM.

Chapter -2
SRS-
Software Requirement Specification

2. Software Requirement Specification (SRS)

2.1 Introduction

This Software Requirement Specification (SRS) defines the functional and non-functional requirements of the CampusNest – Smart School ERP System, including system features, user roles, performance and data management processes. It serves as a guide for the design, development and testing phases, ensuring a clear understanding of system behavior and project objectives among developers, testers and administrators. The document helps reduce errors and ambiguity and will be updated as the system evolves.

2.2 Overall Description

2.2.1 Product Perspective

The CampusNest – Smart School ERP System is a web-based application designed to efficiently manage and streamline academic and administrative processes. It integrates a React.js frontend with a Node.js backend and a MongoDB database to securely handle student, course, attendance and result data. The system supports role-based access for Admin, Faculty and Students, enabling secure data management and easy access to academic information.

2.2.2 Product Functions

- Secure login for Admin, Faculty and Students using credentials.
- Admin dashboard to manage faculty, students, courses, attendance and results.
- Faculty can enter and update student marks.
- Students can view their academic results and performance.
- Generate and export result reports (PDF optional).
- Search and filter results by student or course.

2.2.3 User Characteristics

- **Admin:** Manages faculty, students, courses, attendance, results and reports.
- **Faculty**: Enters marks, manages attendance and updates records.
- **Students:** Can view their own academic performance; read-only access.
- Users are assumed to have basic computer literacy and familiarity with web-based applications.

2.2.4 General Constraints

- Requires internet for web deployment (if hosted online).
- Compatible with desktop or laptop systems running Windows or Linux.
- Requires modern web browser (Chrome, Edge, Firefox, etc.).
- Limited to the features implemented; advanced analytics, mobile app, or notification features are not included in the current version.

2.3 Special Requirements (Software/Hardware – if any)

Software:

- Node.js, Express.js, React.
- HTML, CSS, Bootstrap (for front-end design).
- VS Code or any Python-supported IDE.
- MongoDB Workbench .

Hardware:

- Desktop/Laptop (Windows or Linux OS).
- Minimum: Intel Core i3 Processor, 4 GB RAM.
- 500 GB or more Hard Disk Space.
- Stable Internet Connection (for web access if deployed online).

2.4 Functional Requirements

2.4.1 Module 1: Authentication Module

- Provides secure login for Admin, Faculty and Students.
- Validates credentials and redirects based on role.

2.4.2 Module 2: Student Management

- Admin can add, update and delete student details.
- Supports storing and searching student records.

2.4.3 Module 3: Course Management

- Admin can add, update, and manage course details.
- Supports storing and searching student records.

2.4.4 Module 4: Result Management

- Admin can enter marks and calculate grades automatically.
- Ensures accurate and error-free data handling.

2.4.5 Module 5: Student Result View

- Students can log in to view their own academic performance.
- Displays subject-wise marks, grades, and overall result summary.

2.4.6 Module 6: Report Generation

- Admin can generate and view reports.
- Reports can be viewed, printed, or exported as PDF files.

2.4.7 Module 7: Faculty Management

- Admin can add, update and manage faculty details.
- Maintains faculty records and subject assignments.

2.4.8 Module 7: Attendance Management

- Faculty can record and update student attendance.

- Students can view their attendance details.

2.5 Design Constraints

- Must run in web browsers with responsive interface.
- Should function smoothly on local or hosted servers.
- Database connections must remain secure and stable.
- Only Admins can modify or delete data.

2.6 System Attributes

- **Reliability:** High, as all data is validated before entry.
- **Availability:** Accessible anytime through a web browser (if hosted).
- **Security:** Role-based authentication ensures restricted access.
- **Usability:** Simple and user-friendly interface for Admin, Faculty and Students.
- **Maintainability:** Modular React structure allows easy updates and improvements.

2.7 Other Requirements (If Any)

- Logging of all admin activities (add, update, delete actions).
- Option to generate printable or downloadable result reports.
- Future integration with email/SMS notification system.
- Possibility of adding graphical performance reports in future updates.

Chapter-3
System Design (Functional Design)

3. System Design

3.1 Introduction

Smart School ERP System. It explains how different modules interact to manage, process and display academic System design defines the architecture, components and data flow of the CampusNest –and administrative data efficiently. The system follows a client–server architecture, where a React.js-based frontend communicates with a Node.js backend connected to a MongoDB database. The design ensures data security, modularity and ease of use for Admin, Faculty and Students. Its goal is to automate tasks such as attendance, result processing and report generation while maintaining reliability, scalability and secure data handling.

3.2 Assumptions and Constraints

Assumptions:

- Users (Admin, Faculty and Students) have valid login credentials.
- The system runs on a device with internet access and a web browser.
- The application server and database are properly configured.

Constraints:

- Works best on modern web browsers (Chrome, Edge, Firefox).
- Requires a running Node.js server and MongoDB database.
- Only authorized Admin/Faculty can modify data; Students have view-only access.
- The system does not currently include mobile or SMS integration.

3.3 Functional Decomposition

The Student Result Management System is divided into the following major functional modules:

1. **Login System**
   - Validates Admin and Student credentials for secure access.
   - Redirects users to their respective dashboards based on their role.

2. **Student Management**
   - Allows Admin to add, edit, or remove student records.
   - Ensures accurate mapping of students to their respective classes or courses.

3. **Course Management**
   - Enables Admin to create and manage course details such as name and code.
   - Links each course to the appropriate class for organized result entry.

4. **Result Management**
   - Facilitates entry and modification of student marks and grades.
   - Provides result filtering and summary view by name, roll number, or course.

5. **Student Result View**
   - Allows students to securely log in and view their individual results.
   - Displays detailed marks, grades, and overall performance per exam.

6. **Report Generator**
   - Generates printable and downloadable reports of student performance.
   - Supports exporting result summaries for record-keeping or official use.

3.4 Description of Program:

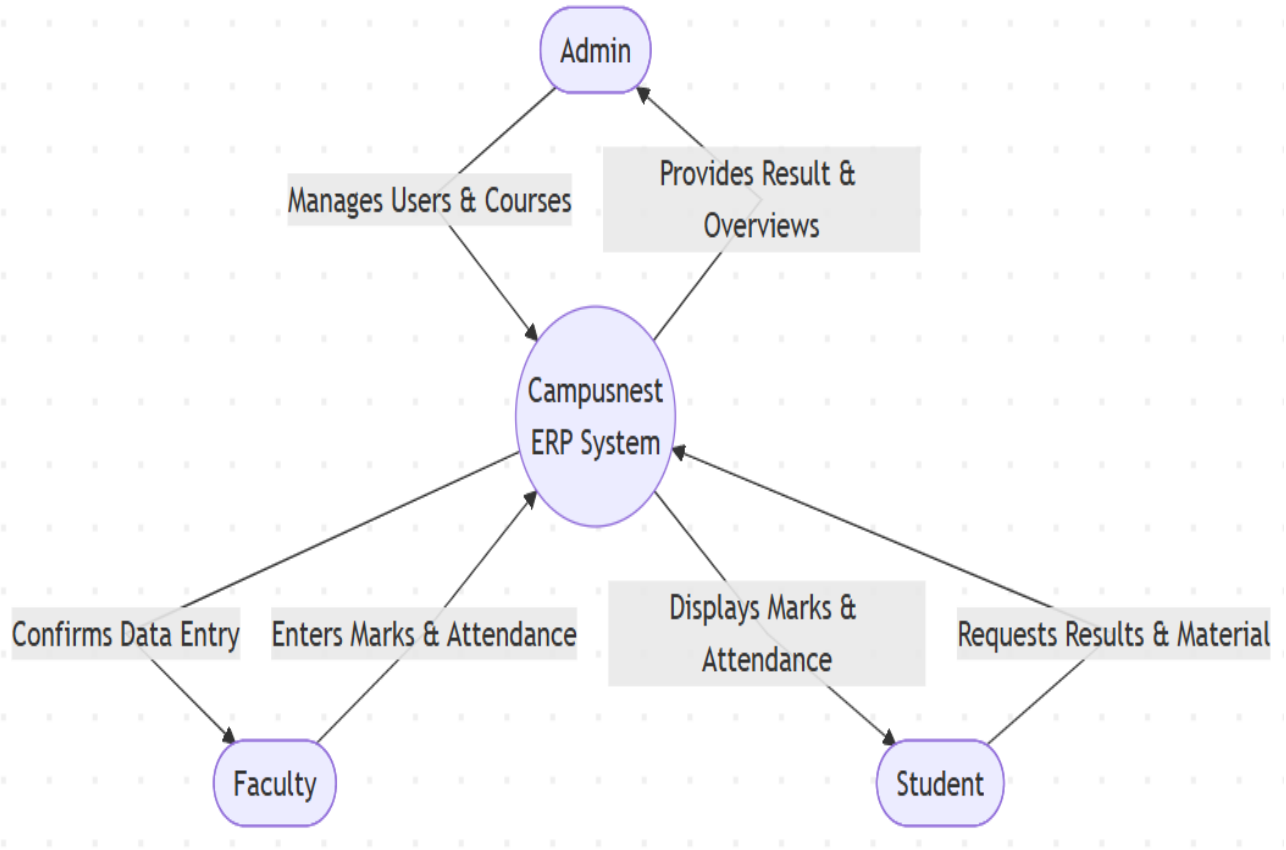
3.4.1 Context Flow Diagram(CFD)

Entities:

- Admin – Manages students, courses, and results.
- Student – Views individual academic results.
- Database (MongoDB) – Stores all system data securely.

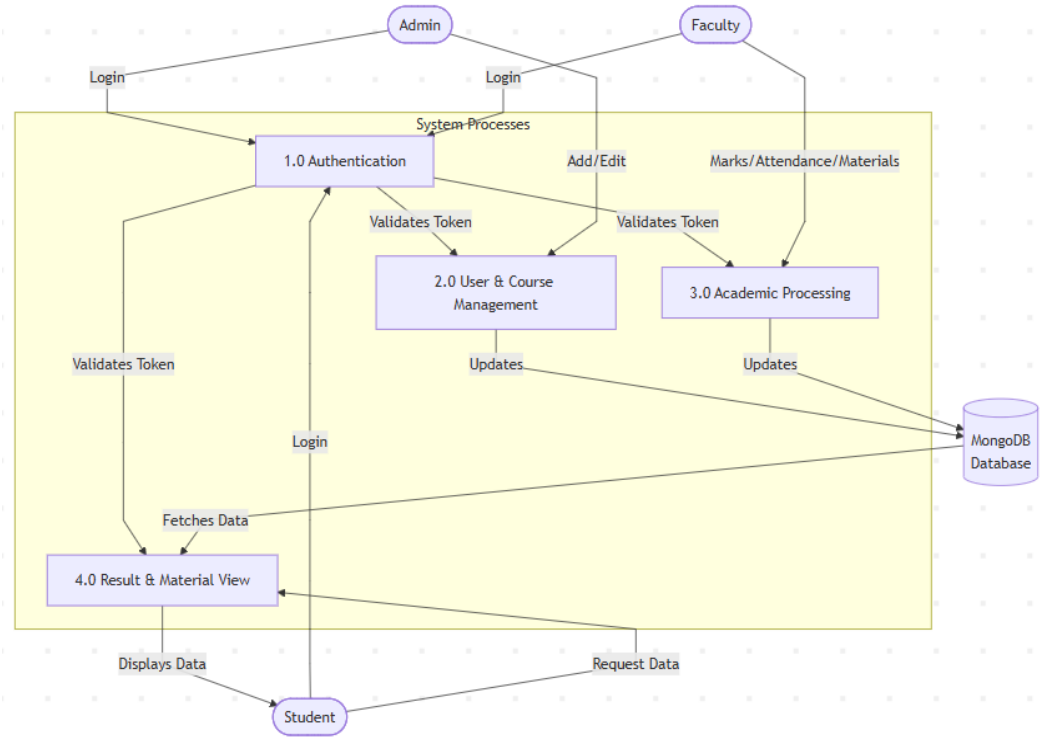
System Block:

- Authentication Module – Validates user credentials.
- Student & Course Management – Handles academic record maintenance.
- Result Processing – Manages mark entry and grade calculation.
- Report Generation – Prepares printable reports and summaries.

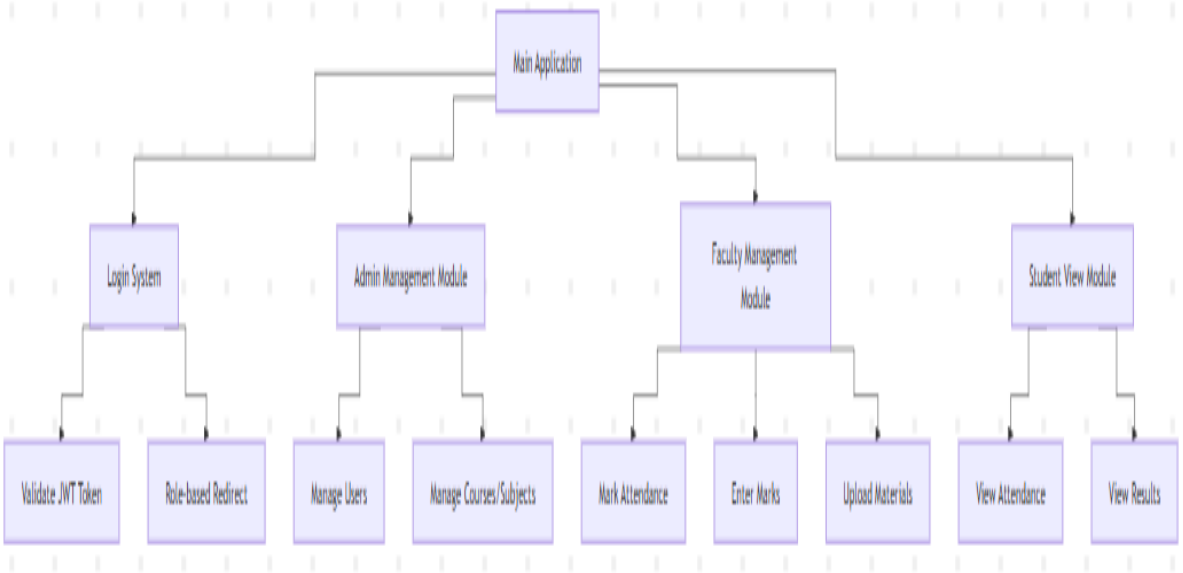


3.4.2 Data Flow Diagrams (DFDs)

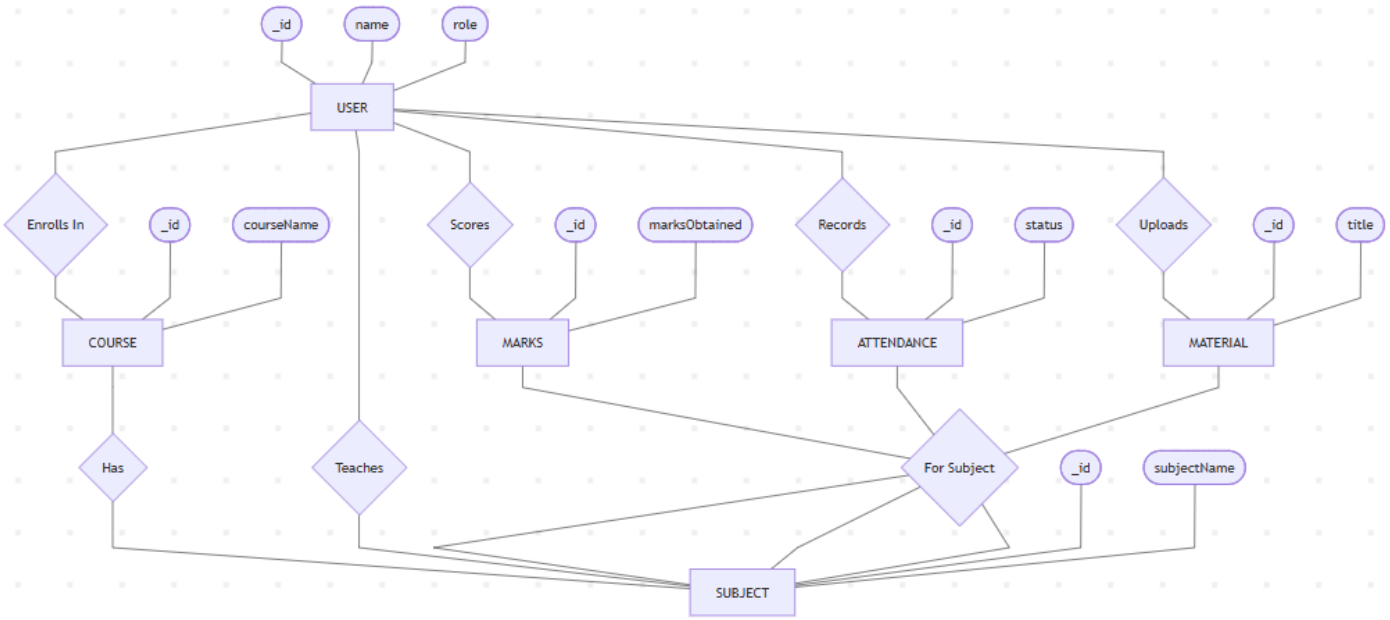
- **Level 0**



- **Level 1**



- **Level 2**


_id
name
role
USER
Enrolls In
_id
courseName
Scores
_id
marksObtained
Records
_id
status
Uploads
_id
title
COURSE
MARKS
ATTENDANCE
MATERIAL
Has
Teaches
For Subject
_id
subjectName
SUBJECT

3.5 Description of Components

3.5.1 Functional Component 1: Authentication

- Purpose: Validate Admin, Faculty, and Student credentials for secure access.
- Tools: Node.js, Express, JSON Web Tokens (JWT), React forms, MongoDB
- Input: Email and password..
- Output: Authorized access to dashboard.
- Challenges: Ensuring security and preventing unauthorized access.

3.5.2 Functional Component 2: Student Management

- Purpose: Manage all student records within the system.
- Tools: Node.js, React, MongoDB.
- Input: Student roll number, subject, date, marks, and files.
- Output: Updated student database.
- Challenges: Ensuring accuracy, managing file uploads, and preventing data loss..

3.5.3 Functional Component 3: Course Management

- Purpose: Maintain course information linked to student records.
- Tools: React, MongoDB.
- Input: Course name, code, and related class.
- Output: Structured course list for result entry and display.
- Challenges: Maintaining consistency between student and course data.

3.5.4 Functional Component 4: Result Management

- Purpose: Handle mark entry, grade calculation, and result display.
- Tools: React, MongoDB.
- Input: Marks for each student per course.
- Output: Computed grades and stored results.
- Challenges: Ensuring accuracy and preventing data loss.

3.5.5 Functional Component 5: Student Result View

- Purpose: Allow students to securely view their individual academic results through the web interface.
- Tools: React , HTML, CSS, Bootstrap, MongoDB.
- Input: Student roll number or login credentials.
- Output: Display of student's marks, grades, and total performance for each course.
- Challenges: Ensuring secure access so only the respective student can view their data and maintaining accurate synchronization between student and course tables.

3.5.6 Functional Component 6: Report Generation

- Purpose: Generate summary reports and export results.
- Tools: React, HTML/CSS, PDF generation library (optional).
- Input: Result and student data from the database.
- Output: Printable and downloadable reports.
- Challenges: Maintaining consistent report formatting.

Chapter-4
Database Design

4. Database Design

4.1 Introduction

Database design is one of the most critical stages in the development of the Campusnest ERP System. It defines how data will be stored, organized, and accessed efficiently. A well-designed database ensures data integrity, minimizes redundancy, and enables faster data retrieval. The Campusnest database uses MongoDB, a reliable and scalable NoSQL database management system. The design leverages schemas and referenced documents to ensure data consistency and efficient access. Each collection represents a realworld entity such as User, Course, Subject, Marks, or Attendance..

4.2 Purpose and Scope

Purpose:

- To store and manage structured data related to students, faculty, courses, examinations, attendance, and results.
- To provide secure, organized, and efficient data handling for academic institutions.

Scope:

- Centralized database accessible through the web-based backend API..
- **Stores information including:**
  - User details (name, email, role, contact, course)
  - Course and Subject information (course name, code)
  - Attendance records (date, status, linked student and subject)
  - Result data (marks obtained, total, linked student and subject)
  - Study Materials and Announcements.

- **The system can later be extended for:**
  - Online deployment with multiple user access.
  - Integration with institutional databases.
  - Result report export in PDF format.

4.3 Table Definition

4.3.1 Table: User

Field Name	Data Type	Description
id	INT	Unique ID for each class
class_name	VARCHAR(100)	Name of the class

4.3.2 Table: Course

Field Name	Data Type	Description
id	ObjectId	Unique ID for each course
name	String	Course name
course_code	String	Code of the course
duration	String	Total duration of the course

4.3.3 Table: Subject

Field Name	Data Type	Description
id	ObjectId	Unique ID for each subject
subjectName	String	Name of the subject
subjectCode	String	Subject code
course	ObjectId	Reference to Course collection
faculty	ObjectId	Reference to Faculty User

4.3.4 Table: Marks

Field Name	Data Type	Description
_id	ObjectId	Unique ID for the mark entry
student	ObjectId	Reference to Student User
subject	ObjectId	Reference to Subject
marksObtained	Number	Marks scored by student
totalMarks	Number	Total marks possible
examType	String	Type of exam (Midterm, Final)

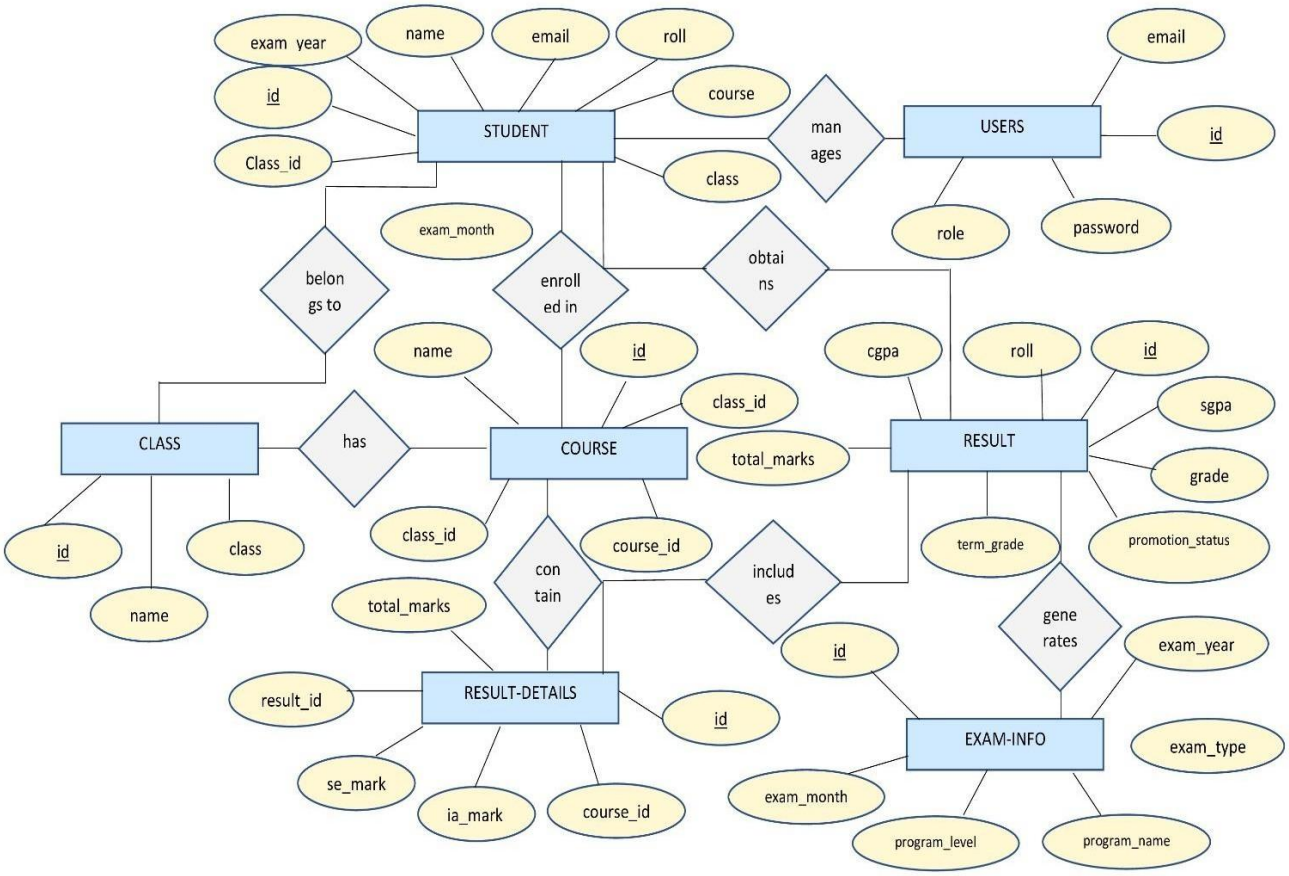
4.3.5 Table: Attendance

Field Name	Data Type	Description
_id	ObjectId	Unique ID for attendance record
student	ObjectId	Reference to Student User
subject	ObjectId	Reference to Subject
date	Date	Date of the class
status	sibject	Present or Absent

4.3.6 Table: Material

Field Name	Data Type	Description
id	ObjectId	Unique ID for material
name	String	Title of the material
roll	String	File path/URL for download
course	ObjectId	Reference to Subject
email	ObjectId	Reference to Faculty User

4.4 ER Diagram



Chapter-5
Detailed Design

5. Detailed Design

5.1 Introduction

The detailed design of the Campusnest ERP System focuses on the internal structure and logic of each functional module. It defines how data flows between components such as login, user management, course management, and academic processing. Each module is designed to perform a specific function—like managing students, handling course details, entering marks, or scheduling classes—ensuring modular development, easy debugging, and future scalability. The design emphasizes security, data integrity, and a user-friendly interface for both administrators, faculty, and students.

5.2 Structure of the Software Package (Structure Chart)

```
Main Application
│
├── Login System
│   ├── Validate User Credentials (Admin/Student)
│   └── Redirect to Respective Dashboard (Admin/Faculty/Student)
│
├── Admin Management Module
│   ├── Add / Edit / Delete Faculty & Student Records
│   └── Manage Courses and Subjects
│
├── Faculty Management Module
│   ├── Add and Manage Attendance Data
│   └── Enter / Update Marks and Grades
│
├── Report Generation Module
│   ├── Generate Printable / Downloadable Reports
│   └── Export Results for Academic Records
│
└── Student Result View Module
    ├── Authenticate Student Login
    └── Display Individual Academic Results Securely
```

5.3 Modular Decomposition of the System

5.3.1 Module 1: Login System

5.3.1.1 Inputs

- User credentials (email and password).
- User role (Admin, faculty or Student).

5.3.1.2 Procedural Details

- Validates entered credentials against the MongoDB database.
- Compares hashed passwords using bcrypt.

5.3.1.3 File I/O Interfaces

- Reads user data from the database user collection.

5.3.1.4 Outputs

- Successful login → Dashboard access with JWT token.
- Invalid login → Error message displayed.

5.3.1.5 Implementation Aspects

- Input validation and NoSQL injection protection
- Use of JWT for secure login state management.

5.3.2 Module 2: Faculty Academic Module

5.3.2.1 Inputs

- Student IDs, Subject IDs, Marks, Attendance Status..

5.3.2.2 Procedural Details

- Faculty selects subject and adds attendance or marks for enrolled students.
- Changes are updated in the Marks and Attendance collections in MongoDB.

5.3.2.3 File I/O Interfaces

- Read/Write operations on Marks and Attendance collections.

5.3.2.4 Outputs

- Confirmation of record addition, modification, or deletion.
- Updated lists displayed on the faculty dashboard.

5.3.2.5 Implementation Aspects

- Input validation to avoid duplicate or invalid range entries.
- Error handling for database operation failures.

5.3.3 Module 3: Course Management Module

5.3.3.1 Inputs

- Course details (Course Name, Class).

5.3.3.2 Procedural Details

- Admin adds, updates, or deletes course records.
- Each course is linked to multiple students and results.

5.3.3.3 File I/O Interfaces

- Read/Write operations on the course table.

5.3.3.4 Outputs

- Confirmation message for successful course management operations.
- Updated course list shown in the admin dashboard.

5.3.3.5 Implementation Aspects

- Maintains foreign key relationships with student and result tables.
- Input validation for unique course codes.

5.3.4 Module 4: Result Management Module

5.3.4.1 Inputs

- Student roll number, course, and obtained marks.

5.3.4.2 Procedural Details

- Admin enters or updates marks for each student.
- Calculates total and grade automatically.

5.3.4.3 File I/O Interfaces

- Read/Write operations on the result table.

5.3.4.4 Outputs

- Stored and retrievable result records.
- Display of computed grades and totals.

5.3.4.5 Implementation Aspects

- Input validation for marks range.
- Error handling for duplicate or missing entries.

5.3.5 Module 5: Student Result View Module

5.3.5.1 Inputs

- Student login credentials (Roll number, password).

5.3.5.2 Procedural Details

- Validates login credentials.
- Retrieves and displays results for the specific student from the database.

5.3.5.3 File I/O Interfaces

- Read operations on the result table.

5.3.5.4 Outputs

- Display of marks, grades, and total performance.

5.3.5.5 Implementation Aspects

- Ensures secure access — only the logged-in student can view their data.
- Responsive web design for ease of viewing on different devices.

5.3.6 Module 6: Report Generation Module

5.3.6.1 Inputs

- Selected student or course for report generation.

5.3.6.2 Procedural Details

- Fetches data from the database and formats it into printable or downloadable reports.
- Allows filtering by roll number, course, or academic year.

5.3.6.3 File I/O Interfaces

- Reads data from student, course, and result tables.
- Optionally exports report as PDF.

5.3.6.4 Outputs

- Printable report or downloadable PDF file.

5.3.6.5 Implementation Aspects

- Uses React and report generation libraries.
- Maintains report formatting consistency.

Chapter-6
Program Code Listing

6. Program Code Listing

6.1 Database Connection

```
const mongoose = require('mongoose');

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGODB_URI);
        console.log(`MongoDB Connected: ${conn.connection.host}`);
    } catch (error) {
        console.error(`Error: ${error.message}`);
        process.exit(1);
    }
};
```

6.2 Authorization / Authentication

```
const authUser = async (req, res) => {
    const { email, password } = req.body;

    const user = await User.findOne({
        $or: [ { email: email }, { name: email }, { enrollmentNumber: email } ]
    });

    if (user && (await user.matchPassword(password))) {
        res.json({
            _id: user._id,
            name: user.name,
            email: user.email,
            role: user.role,
            token: generateToken(user._id, user.role),
        });
    } else {
        res.status(401).json({ message: 'Invalid credentials' });
    }
};

const registerUser = async (req, res) => {
    const { name, email, password, role } = req.body;
    const userExists = await User.findOne({ email });

    if (userExists) {
        return res.status(400).json({ message: 'User already exists' });
    }
```

```
const user = await User.create({ name, email, password, role: role || 'Student'
 });

if (user) {
   res.status(201).json({
       _id: user._id,
       name: user.name,
       email: user.email,
       role: user.role,
       token: generateToken(user._id, user.role),
   });
} else {
   res.status(400).json({ message: 'Invalid user data' });
}
 };
```

6.3 Home and Dashboard

```
const express = require('express');

const router = express.Router();

const { protect, admin } = require('../middleware/authMiddleware');

const { getAdminDashboardStats } = require('../controllers/adminController');


// Only Admins can access this dashboard route

router.get('/dashboard-stats', protect, admin, getAdminDashboardStats);
```

6.4 Student Management

```
const approveStudent = async (req, res) => {

   try {

       const student = await

User.findById(req.params.id);

       if (student && student.role === 'Student') {

           student.isApproved = true;

           await student.save();

           res.json({ message: 'Student approved

successfully' });

       } else {
```

```
            res.status(404).json({ message: 'Student not
found' });
        }
    } catch (error) {
        res.status(500).json({ message: error.message
});
    }
};
```

6.5 Course Management

```
const createCourse = async (req, res) => {
    try {
        const { code, name, department, credits, semester } = req.body;
        const courseExists = await Course.findOne({ code });

        if (courseExists) {
            return res.status(400).json({ message: 'Course with this code already exists' });
        }

        const course = await Course.create({ code, name, department, credits, semester });
        res.status(201).json(course);
    } catch (error) {
        res.status(500).json({ message: error.message });
    }
};
```

6.6 Result Management

```
const addOrUpdateMarks = async (req, res) => {
    try {
        const { studentId, courseId, marksType, marksObtained, totalMarks, remarks }
= req.body;
        const facultyId = req.user._id;

        const newMark = await Marks.create({
            student: studentId,
            course: courseId,
            faculty: facultyId,
            marksType,
            marksObtained,
            totalMarks,
            remarks,
            dateRecorded: new Date()
        });

        res.status(201).json({ message: 'Marks recorded successfully', mark: newMark
```

```
    });
        } catch (error) {
            res.status(500).json({ message: error.message });
        }
    };
```

6.7 View Results

```
const getMyMarks = async
(req, res) => {
  try {
      const marks = await
Marks.find({ student:
req.user._id })
          .populate('course',
'name code credits')
          .populate('faculty',
'name')
          .sort({
dateRecorded: -1 });

      res.json(marks);
  } catch (error) {
      res.status(500).json({
message: error.message });
  }
};
```

6.8 Exam Information

```
@ const marksSchema = new
mongoose.Schema({
  student: { type:
mongoose.Schema.Types.ObjectId, ref: 'User',
required: true },
  course: { type:
mongoose.Schema.Types.ObjectId, ref: 'Course',
required: true },
  marksType: { type: String, required: true,
enum: ['Midterm', 'Final', 'Assignment', 'Quiz',
'Project'] },
```

```
    marksObtained: { type: Number, required: true
  },
    totalMarks: { type: Number, required: true },
  });
```

6.9 Error Handling

```
const notFound = (req, res,
    next) => {
  const error = new
    Error(`Not Found -
    ${req.originalUrl}`);
  res.status(404);
  next(error);
    };

const errorHandler = (err, req, res, next) => {
  const statusCode = res.statusCode === 200 ?
    500 : res.statusCode;
  res.status(statusCode);
  res.json({
     message: err.message,
     stack: process.env.NODE_ENV ===
    'production' ? null : err.stack,
  });
    };
```

6.10 Application Entry Point

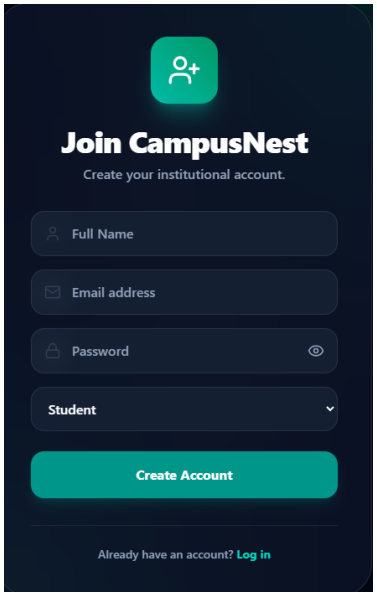
```
const express = require('express');
const dotenv = require('dotenv');
const connectDB = require('./config/db');

dotenv.config();
connectDB();

const app = express();
app.use(express.json());
```

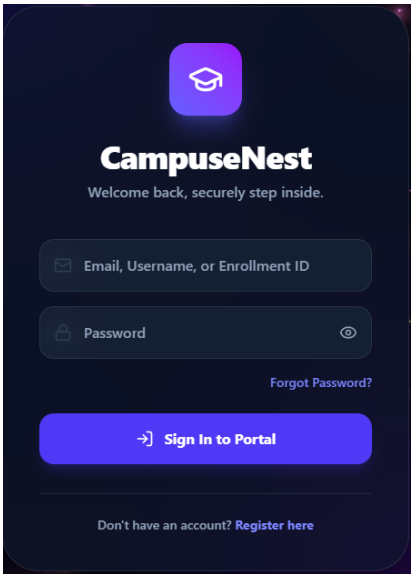
Chapter-7
User Interface

7. User Interface

7.1 Login Page

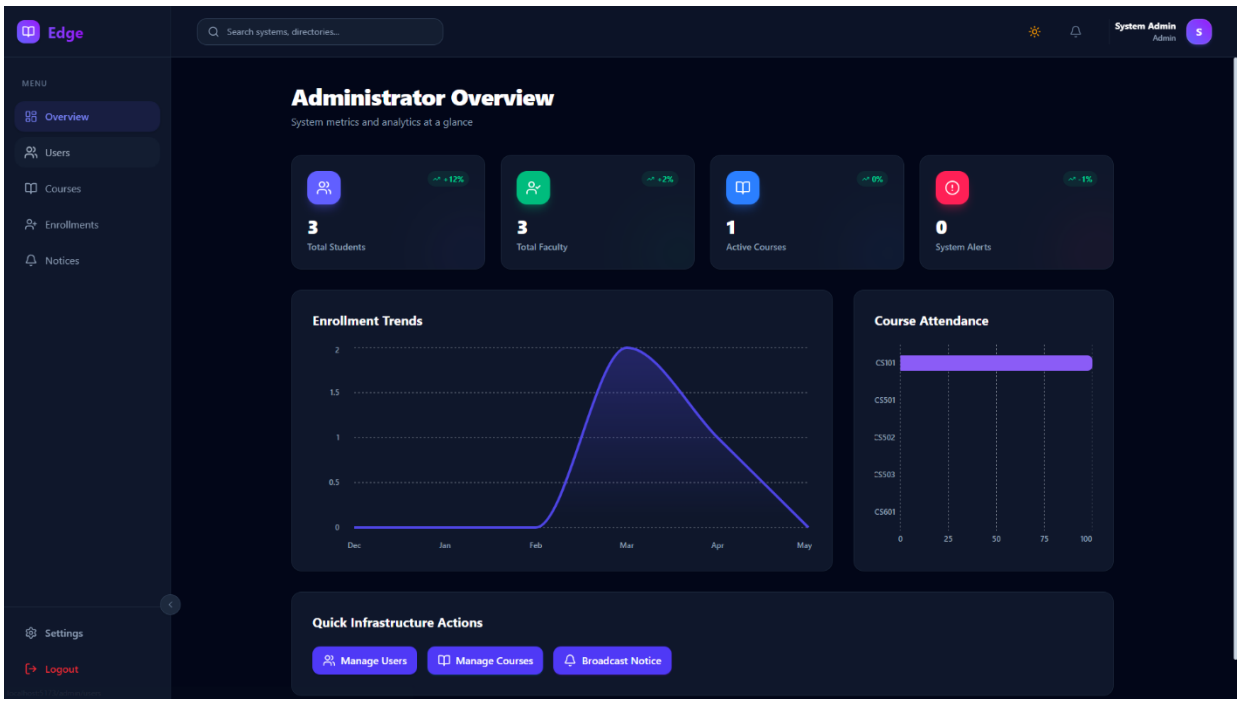
Login Page The Login Page allows users to securely log in using their email and password. Invalid login attempts trigger flash messages to notify the user. The page uses Framer Motion for smooth entrance animations.





7.2 Admin Dashboard

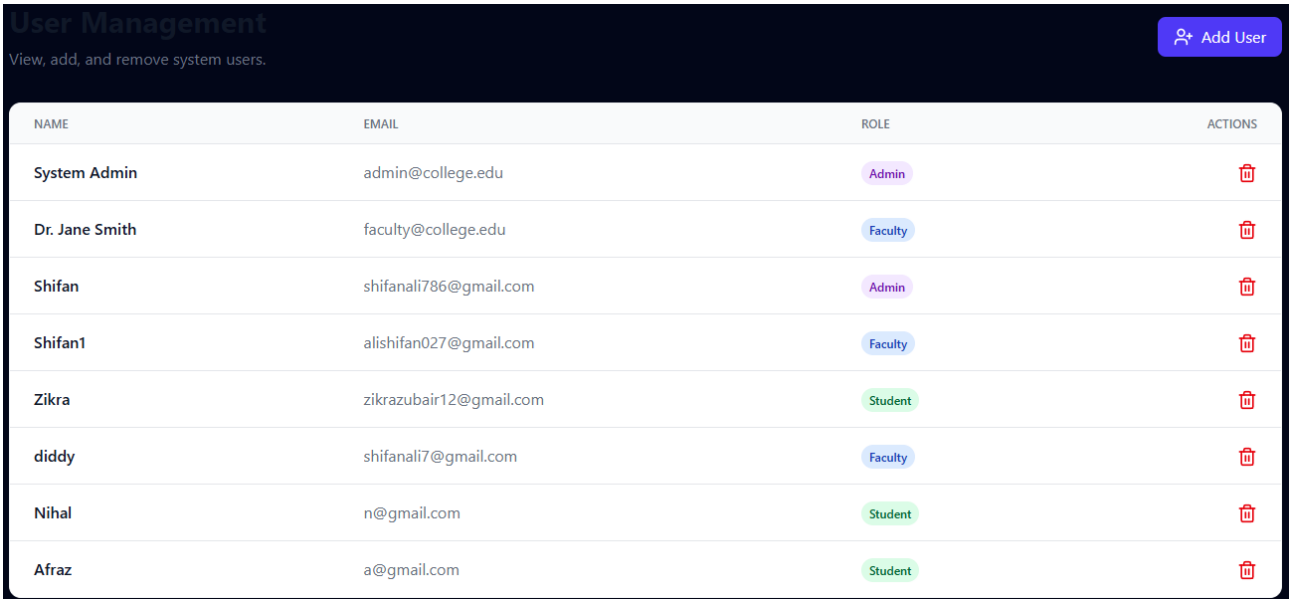
The Home Page welcomes the user and displays rolespecific options. Admins can manage students, courses, subjects, and view total user metrics. Faculty can manage attendance, marks, and upload materials. Students view their personal info, grades, and attendance summaries.



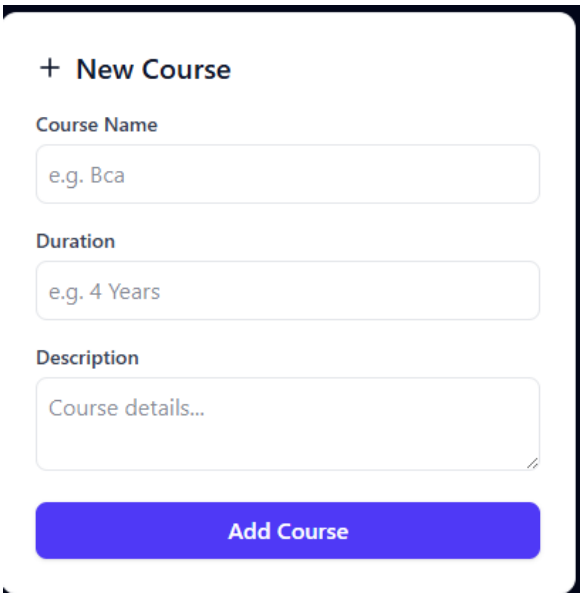
7.3 Data Management

Student, course, and result data are submitted through styled forms. Admin can add or edit information securely. Data is fetched asynchronously via Axios and displayed in cleanly structured tables

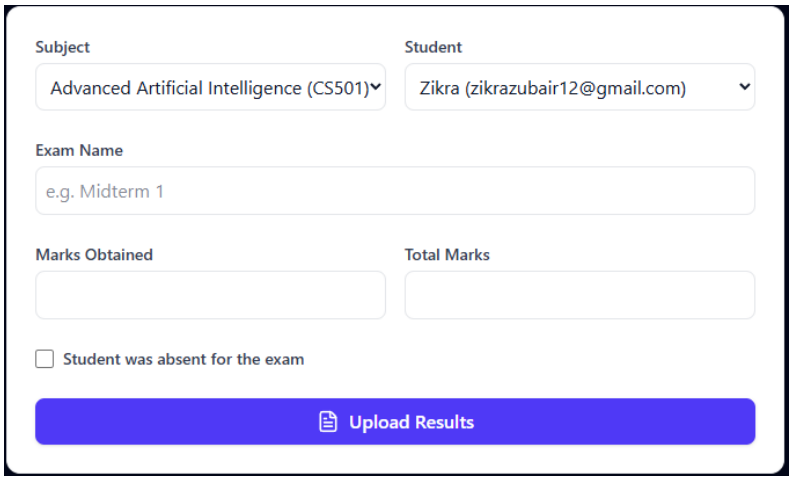
Add Student



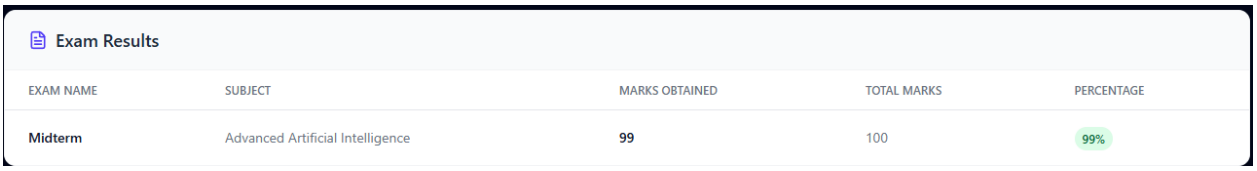
Add Course



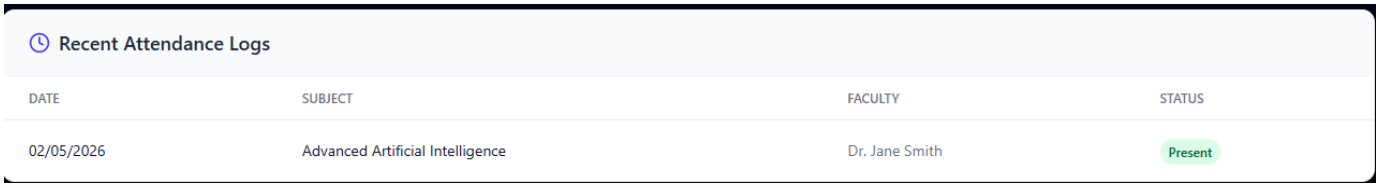
Add Result



View Result

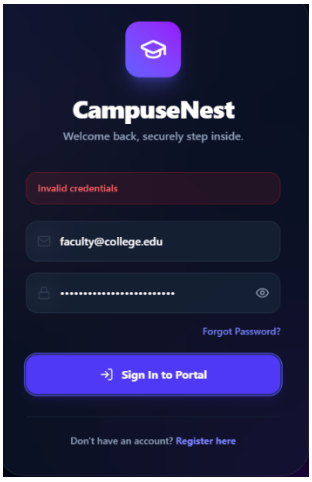


View Attendance



7.4 Validation

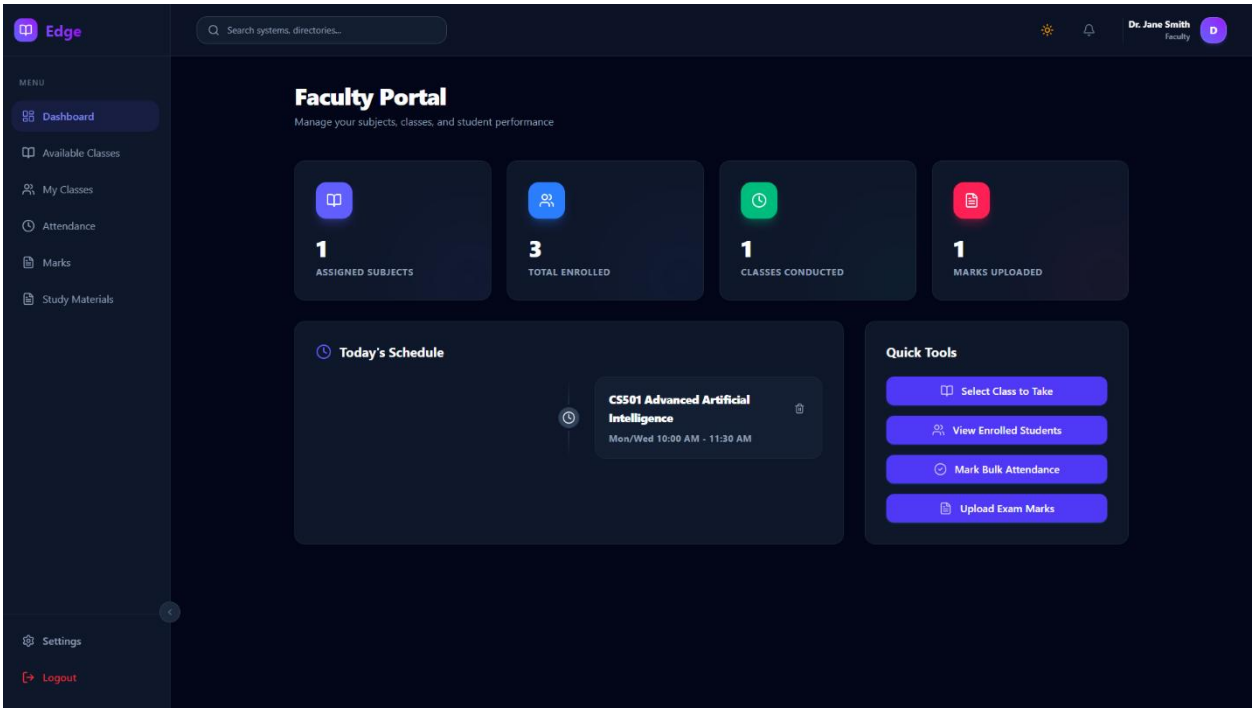
The system checks for valid login credentials, unique email addresses, proper numeric inputs for marks, and ensures all required fields are filled before submitting forms to the backend API..



7.5 View

7.5.1 Faculty Dashboard

Admins can see all student details and results grouped by class, including course-wise marks, grades, SGPA, CGPA, and promotion status.



Chapter-8
Testing

8. Testing

8.1 Introduction

Software testing is a crucial phase in the software development life cycle that ensures the system functions according to its specified requirements. For the Campusnest ERP System, testing was carried out at multiple levels — unit testing, integration testing, and system testing — to verify the accuracy, reliability, and performance of each functional module such as Login, User Management, Academic Processing, and Student View. During unit testing, each API route and React component was tested individually to ensure its components performed as expected. In integration testing, interactions between the frontend and backend were verified to confirm seamless data flow across the system via Axios. Finally, system testing was conducted to evaluate the complete MERN stack in a real-time environment, ensuring that it operated efficiently, securely, and accurately for administrators, faculty, and students. Edge cases such as invalid input data, missing records, and incorrect credentials were also tested to verify the system's robustness. The main goal was to ensure that Campusnest performs reliably, provides accurate results, and maintains data integrity across all modules..

8.1.1 Unit Testing

Each module of the system was tested separately to ensure proper functionality and data accuracy.

- **Login Module:**
  Tested for both admin and student logins using valid and invalid credentials to ensure secure authentication and correct redirection to their respective dashboards.

- **Student Management Module:**
  Verified adding, updating, and deleting student details. Tested for duplicate entries, missing fields, and validation of roll numbers.

- **Course Management Module:**
  Ensured courses could be added, modified, and deleted successfully. Validated that each course is correctly associated with a class or semester.

- **Result Management Module:**
  Checked the accuracy of result entry, marks calculation, and updating of records. Tested for invalid data input and ensured recalculation worked correctly after updates.

- **Student Result View Module:**
  Ensured that students could log in securely and view their results accurately. Tested data display for different students to confirm privacy and correctness.

- **Report Generation Module:**
  Tested the generation of printable and downloadable reports in PDF format. Verified correct data retrieval and formatting.

- **Tools Used:**
  The system was tested using Python's **unittest framework** for automated testing of key functionalities, along with **manual testing** by running the application in VS Code. Manual testing included logging in as admin and student, adding and editing student and course data, entering exam results, viewing student information and results, and checking alerts and error messages. This ensured the system functions correctly and all validations work as intended.

8.1.2 Integration Testing

Integration testing focused on ensuring that the frontend and backend interacted correctly.

Key Interactions Tested:

- Successful login should store the JWT token and redirect users to their respective dashboards.
- Data added in the Admin Management Module should automatically reflect in the database and subsequent GET requests.
- Marks and attendance entered by faculty should instantly be visible on the Student Result View.

Scenarios Covered:

- Login → Dashboard → Module Access.
- Student & Course Data Integration → Result Calculation.
- Result Update → Report Generation → Student View.

These tests confirmed that the system-maintained data consistency and logical flow between modules.

8.1.3 System Testing

System testing evaluated the complete SRMS as a unified application to verify that all components worked together smoothly and fulfilled the project's functional and non-functional requirements.

Key Functional Areas Tested:

- **Login and Authentication:** Verified secure login for admin and student users, including validation for incorrect credentials.
- **Student Management:** Confirmed that administrators could add, update, and delete records successfully.
- **Course Management:** Ensured that all course-related information was displayed correctly and linked with student data.
- **Result Management:** Tested result entry, grade calculation, and retrieval accuracy.
- **Student Result View:** Checked accurate display of student results based on their roll numbers and login credentials.
- **Report Generation:** Verified the generation of well-formatted printable reports for both admin and student use.

System Behaviors Validated:

- Data integrity across modules.
- Real-time result updates after admin modifications.
- Proper error messages for invalid inputs.
- Secure data access (students can only view their own results).
- Stable system performance during concurrent access.

Result:

The Student Result Management System performed efficiently and accurately during testing. It met all functional, performance, and usability requirements, confirming that the system is ready for deployment in an educational environment.

8.2 Test Reports

Test Case ID	Description	Expected Result	Status
TC_001	Login with valid credentials	Redirects to dashboard with token	Passed
TC_002	Login with invalid credentials	Displays “Invalid credentials” error	Passed

TC_003	Admin adds new student	Student added successfully to DB	Passed
TC_004	Faculty enters attendance	Attendance saved and calculated	Passed
TC_005	Student views marks	Displays correct individual result	Passed
TC_006	Faculty uploads material	File stored and URL saved to DB	Passed
TC_007	Delete student record	Student data deleted	Passed
TC_008	Attempt to view unauthorized route	Access denied, redirects to login	Passed

Chapter-9
Conclusion

9. Conclusion

The **Campusnest ERP** System provides an efficient, secure, and automated platform for managing academic records within educational institutions. It simplifies result processing, attendance management, and material sharing through an intuitive web-based interface accessible to administrators, faculty, and students..

The system ensures data accuracy, security, and transparency by integrating functionalities such as user registration, course management, marks entry, and secure data retrieval. It eliminates manual errors commonly found in paper-based systems and allows faculty to quickly update student performance records.

By providing role-based access, administrators have complete control over system data, while students can securely view their individual results anytime. The use of the MERN stack ensures structured storage, fast execution, and quick access to academic information.

Overall, **Campusnest** enhances institutional productivity by automating repetitive tasks, minimizing workload, and ensuring that data remains consistent and reliable. The system can easily be scaled to support more courses, semesters, and departments as the institution grows.

Chapter -10
Limitations

10. Limitations

- **Internet Dependency:**
  Online access is required for real-time updates and data synchronization between the React client and Node.js server.

- **Single Institution Scope:**
  The system is currently designed for use within one college or specific departments, lacking a multi-tenant architecture..

- **Limited Analytics:**
  The system does not yet provide advanced analytical tools for performance trends or visual charting comparisons.

- **Manual Data Entry:**
  Student and result data must be entered manually, which may take time for large batches without bulk import tools.

- **No Mobile Application:**
  Currently optimized only for desktop and mobile browsers via responsive CSS; a dedicated native mobile version is not yet implemented.

Chapter -11
Scope of Enhancement

11. Scope for Enhancement

- **Automated Result Analysis:**
  Integrate data visualization tools (like Recharts) to display performance trends and subject-wise statistics graphically.

- **Email/SMS Notifications:**
  Notify students automatically when results or new attendance warnings are published.

- **Mobile App Integration:**
  Develop Android and iOS versions using React Native for better accessibility and push notifications..

- **Import/Export Functionality:**
  Allow bulk uploading of student and marks data from Excel or CSV files.

- **AI-Based Insights:**
  Implement algorithms to predict academic performance or identify students at risk based on attendance trends.

- **Offline Access Mode:**
  Allow limited system use during network outages with local data synchronization later via Service Workers.

Chapter -12
Abbreviations and Acronyms

12. Abbreviations and Acronyms

Acronym	Full Form
AI	Artificial Intelligence
Admin	Administrator
API	Application Programming Interface
CGPA	Cumulative Grade Point Average
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSS3	Cascading Style Sheets version 3
CFD	Context Flow Diagram
CSV	Comma-Separated Values
DB	Database
DFD	Data Flow Diagram
React	Micro web framework for Python
HTML	HyperText Markup Language
HTML5	HyperText Markup Language version 5
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
MongoDB	My Structured Query Language (relational database management system)
PDF	Portable Document Format
PG	Postgraduate
RA	Research Assistant (if applicable)
SGPA	Semester Grade Point Average
SRMS	Student Result Management System
SMS	Short Message Service
SQL	Structured Query Language
UG	Undergraduate
UI	User Interface
URL	Uniform Resource Locator
VS Code	Visual Studio Code

Chapter -13
Bibliography / References

13. Bibliography / References

1. **Bootstrap :**
   https://getbootstrap.com/

2. **ChatGPT :**
   https://chatgpt.com/

3. **React (Python Web Framework) :**
   https://React.palletsprojects.com/

4. **Gemini :**
   https://gemini.google.com/

5. **MongoDB Workbench :**
   https://www.mongoDB.com/products/workbench/

6. **Python :**
   https://www.python.org/

7. **VS Code (Visual Studio Code) :**
   https://code.visualstudio.com/

8. **YouTube :**
   https://www.youtube.com/